

Operators and Expressions

Operators (Agarwal & Bansal, 2023)

- **Overview:** "Operators are symbols used to compute and compare values and test multiple conditions." Java supports several categories of operators.
- **Arithmetic Operators:** Used to perform all mathematical operations, where an operand is the value to be computed.
 - **Addition (+):** Adds an operand ($x = y + z$).
 - **Subtraction (-):** Subtracts the right operand from the left operand ($x = y - z$).
 - **Multiplication (*):** Multiplies the operands ($x = y * 3$).
 - **Division (/):** Divides the left operand by the right operand ($x = y/z$).
 - **Modulus (%):** Calculates the remainder of an integer division ($x = y \% z$).
- **The + Operator with Numeric Data Types:** Adding operands of the primitive numeric data type results in a primitive numeric data type. Since the byte data type can hold a range of -128 to +127, Java automatically converts any calculations of numeric data results to the `int` data type.
- **The + Operator with the String Data Type:** If both operands are strings, the `+` operator joins them to display as a single output; if one of the operands is a string object, the second operand is converted into a string before joining.

Assignment Operators

- An assignment operator assigns the specific value, variable, and/or functions to another variable.
- **Assignment (=):** Assigns the value of the right operand to the left operand ($x = y$).
- **Add and Assignment (+=):** Adds the operands and assigns the result to the left operand ($x += y$).
- **Subtract and Assignment (-=):** Subtracts the right operand from the left operand and stores the result in the left operand ($x -= y$).
- **Multiply and Assignment (*=):** Multiplies the operand by the right operand and stores the result in the left operand ($x *= z$).
- **Divide and Assignment (/=):** Divides the left operand by the right operand and stores the result in the left operand ($x /= z$).
- **Modulus and Assignment (%=):** Divides the left operand by the right operand and stores the remainder in the left operand ($x \% = z$).
- Note: `x += y` is equivalent to `x = x + y`, which applies to every operator above. Assigning values to more than one variable at the same time is also allowed (e.g., `x = y = 0`).

Unary Operators

- Unlike previous operators, these operators require only one operand.
- **Unary Plus (+):** Represents the positive value ($+ a$).
- **Unary Minus (-):** Represents the negative value ($- a$).
- **Increment (++):** Increases the value of the operand by 1 ($x ++$). Can be used as a prefix (`++invar`) or a suffix (`invar++`).
- **Decrement (--):** Decreases the value of the operand by 1 ($x --$).

- **Logical Complement (!):** Inverts the value of a Boolean variable (!true).

Comparison Operators

- Also called relational operators, these perform all mathematical operations involving comparison between values.
- Includes **Equal to (==)**, **Not Equal to (!=)**, **Greater Than (>)**, **Less Than (<)**, **Greater Than or Equal to (>=)**, and **Less Than or Equal to (<=)**.

Shift Operators

- Work on bits of data by moving the bit pattern to the left or right, and can only be used with integer data types.
- **Right Shift (>>):** The bits of the left operand are shifted right by the number specified as the value for the second operand ($x = 9 \gg 2$).
- **Left Shift (<<):** Shifts bits to the left, filling zeros at the right ($x = 9 \ll 2$).
- **Right Unsigned Shift (>>>):** Works the same as the right shift, but 0 is used to replace the bits that are shifted ($x = -10 \ggg 3$).

Bitwise Operators

- Perform operations bit-by-bit and can be applied to byte, int, short, long, and char data types.
- **AND (&):** Results in a 1 if both bits are 1; any other combination results in 0.
- **OR (|):** Results in a 0 when both bits are 0; any other combination results in 1.
- **XOR (^):** Results in a 0 if both bits are of the same value and 1 if the bits have different values.
- **Inversion (~):** Converts all 1 bits to 0's and all 0 bits to 1's (~x).

Logical Operators

- Used to combine the results of Boolean expressions and are limited to Boolean expressions only.
- **Logical AND (&&):** If both operands are non-zero (or true), the expression returns true; otherwise, it returns false.
- **Logical OR (||):** If one or both operands are non-zero, the expression returns true; otherwise, it returns FALSE.

Conditional Operator

- **Ternary ((condition) ? val1 : val2):** Evaluates **val1** if the condition returns true and **val2** if the condition returns false ($x = (y > z) ? y : z$).

Order of Precedence of Operators

- Shows the hierarchy wherein an operation with high precedence is performed before others.
- Precedence from highest to lowest: [], (); +, -, ~, !, ++, --; *, /, %; +, -; <<, >>, >>>; <, <=, >=, >; =, ==, !=; &, ^, |; &&, ||; ?:; =, +=, -=, *=, /=, %=.

Category	Operators
High Precedence	[], ()
Unary	+, -, ~, !, ++, --
Multiplicative	*, /, %
Additive	+, -
Shift	<<, >>, >>>
Relational	<, <=, >=, >
Equality	=, ==, !=
Bitwise	&, ^,
Logical	&&,
Conditional	?:
Assignment	=, +=, -=, *=, /=, %=

Expressions (Agarwal & Bansal, 2023)

- **Overview:** "An expression is a combination of variables, constants, literals, and operators to produce a single value." An expression statement consists of an expression followed by a semicolon (;).
- **Arithmetic Expression:** Returns a numeric value based on the operators and operands used (e.g., `int z = x + y;`).
- **Assignment Expression:** Involves assigning a value to a variable (e.g., `a = a + 4;`).
- **Relational Expression:** Compares values using relational operators (e.g., `boolean result = x < y;`).
- **Logical Expression:** Involves logical operators (e.g., `boolean result = (j < k) && (k < 4);`).
- **Conditional Expression:** Involves the use of the ternary operator (?:) to assign a value based on a condition (e.g., `int n = (l < m) ? 2 : 3;`).